

EC2 Deployment Guide

CIS Technologies — deploying directly onto a single EC2 instance

Docker Compose · Caddy with automatic TLS · Postgres · Redis · systemd · nightly backups

The files are already in your repository.

Everything this guide references lives in `deploy/ec2/` and has been validated:

File	What it does
<code>docker-compose.prod.yml</code>	The whole stack. Only Caddy publishes ports; everything else is on an isolated network.
<code>Caddyfile</code>	TLS certificate issued and renewed automatically; routes <code>/api/*</code> to the API and the rest to the SPA.
<code>.env.example</code>	Every variable you need, annotated. Copy to <code>.env</code> on the server.
<code>backup.sh</code>	Nightly <code>pg_dump</code> , gzip, optional S3 upload, 14-day rotation.
<code>cis.service</code>	systemd unit so the stack starts on boot.

1 · Is EC2 the right choice here?

	Single EC2	ECS Fargate
Setup effort	An afternoon	A day or two
Cost (small)	~\$20–35/month	~\$110/month
Deploys	SSH in, <code>git pull</code> , rebuild — brief downtime	Rolling, zero downtime
If the box dies	The site is down until you rebuild it	Tasks reschedule automatically
Patching	Yours	AWS handles the host
Backups	Yours — <code>backup.sh</code> plus a tested restore	RDS automated, point-in-time

EC2 is a sound choice for a pilot, an internal tool, or a first paying customer.

Be clear-eyed about what you are accepting: one machine, one disk, and downtime during every deploy and every reboot. Move when you have users who would notice.

Two shapes

Shape	When
A. All-in-one this guide	Postgres and Redis run as containers on the same instance. Cheapest and simplest. You own the backups completely.

B. EC2 + RDS

for real production

Keep the app containers on EC2; move the database to RDS. You get managed backups and point-in-time recovery for roughly \$15–60/month more. Only change: delete the `postgres` service from the compose file and point `DATABASE_URL` at RDS.

2 · Before you start

- ☐ **Clear blocker 1 from the readiness checklist first.** Run `npx prisma migrate dev --name init` locally and commit `apps/api/prisma/migrations/`. The compose file runs `migrate deploy`, which does nothing if that folder is missing — you would get a running app with an empty database.
- ☐ A domain you control, ready to point at an IP.
- ☐ An AWS account, and an SSH key pair created in your chosen region.
- ☐ Blocker 2 is already handled: Caddy does the `/api/*` routing.

Do not skip the migration step.

It is the single most common reason a first deployment comes up, accepts a login, and shows nothing at all.

3 · Launch the instance

Setting	Value
AMI	Ubuntu Server 24.04 LTS (or Amazon Linux 2023)
Type	<code>t3.small</code> minimum · <code>t3.medium</code> if you build images on the box
Storage	30 GB gp3, encrypted, delete-on-termination off
Key pair	Your SSH key
IAM role	Only if using S3 backups or SSM — attach one with <code>s3:PutObject</code> on your bucket

A `t3.micro` will fail.

Building the web image runs Vite and TypeScript, which needs more than 1 GB; the build is killed by the OOM reaper part-way through with a confusing error. Either use `t3.small` with swap (\$5), or build images elsewhere and pull them.

Security group

Port	Source	Why
22 (SSH)	Your IP only	Never <code>0.0.0.0/0</code>
80	<code>0.0.0.0/0</code>	Certificate challenge and the HTTPS redirect
443	<code>0.0.0.0/0</code>	The application

Nothing else. Postgres and Redis are on an internal Docker network and are not reachable from outside the instance — the compose file never publishes their ports.

Elastic IP and DNS

```
aws ec2 allocate-address --domain vpc  
aws ec2 associate-address --instance-id i-xxxx --allocation-id eipalloc-xxxx
```

Then create an **A record** for `cis.example.com` pointing at that IP and wait for it to resolve. Caddy cannot obtain a certificate until DNS is live.

```
dig +short cis.example.com      # must return your Elastic IP before you continue
```

Allocate an Elastic IP before you point DNS at anything.

A stopped instance loses its public IP, and you would have to update DNS on every restart.

4 · Prepare the server

```
ssh -i ~/.ssh/your-key.pem ubuntu@cis.example.com

sudo apt update && sudo apt upgrade -y
sudo apt install -y ca-certificates curl git ufw fail2ban unattended-upgrades

# Docker, official repository
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo $VERSION_CODENAME) stable" \
  | sudo tee /etc/apt/sources.list.d/docker.list
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

sudo usermod -aG docker $USER && newgrp docker
docker compose version
```

Hardening — do this now, not later

```
# Firewall (belt and braces alongside the security group)
sudo ufw default deny incoming && sudo ufw default allow outgoing
sudo ufw allow 22 && sudo ufw allow 80 && sudo ufw allow 443
sudo ufw --force enable

# Disable password and root SSH login
sudo sed -i 's/^##PasswordAuthentication.*/PasswordAuthentication no/' /etc/ssh/sshd_config
sudo sed -i 's/^##PermitRootLogin.*/PermitRootLogin no/' /etc/ssh/sshd_config
sudo systemctl restart ssh

sudo systemctl enable --now fail2ban
sudo dpkg-reconfigure -plow unattended-upgrades # automatic security updates
```

Swap — required on t3.small

```
sudo fallocate -l 2G /swapfile && sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

Without swap, a 2 GB instance will be killed mid-build. This is the difference between "the build is slow" and "the build dies at 80% with no useful message".

5 · Deploy

```
sudo mkdir -p /opt/cis && sudo chown $USER:$USER /opt/cis
git clone https://github.com/YOUR_ORG/YOUR_REPO.git /opt/cis
cd /opt/cis/deploy/ec2

cp .env.example .env
chmod 600 .env
nano .env          # fill in every CHANGE_ME
```

Generate the secrets

```
openssl rand -base64 48      # JWT_ACCESS_SECRET
openssl rand -base64 48      # JWT_REFRESH_SECRET (must differ)
openssl rand -base64 32      # POSTGRES_PASSWORD

# web push keys
docker run --rm node:20-alpine sh -c \
  "npm i -s web-push && node -e \"console.log(require('web-push').generateVAPIDKeys())\""
```

POSTGRES_PASSWORD appears twice

— once on its own and once inside `DATABASE_URL`. They must match, and if the password contains `@` `:` `/` `?` or `#` it must be percent-encoded inside the URL. Easiest is to generate one without them.

Point the Caddyfile at your domain

```
sed -i 's/cis.example.com/YOUR_DOMAIN/' Caddyfile
```

Start it

```
docker compose -f docker-compose.prod.yml up -d --build
```

The first run takes several minutes: it builds both images, starts Postgres, runs the migration to completion, then starts the API and web. Caddy requests a certificate as soon as the domain resolves.

```
docker compose -f docker-compose.prod.yml ps          # all healthy?
docker compose -f docker-compose.prod.yml logs -f api # watch it boot
curl -fsS https://YOUR_DOMAIN/api/v1/health/ready
```

Expect `{"success":true,"data":{"status":"ready","checks":{"database":true,"cache":true}}}`.

6 · Create the first account

Do not run the seed. It creates a demo organisation and four users sharing one password. Register once through the API — this creates the organisation and makes you its OWNER:

```
curl -X POST https://YOUR_DOMAIN/api/v1/auth/register \
  -H 'Content-Type: application/json' \
  -d '{"organizationName":"CIS Technologies","fullName":"Your Name",
    "email":"you@cis.example.com","password":"a-long-strong-password"}
```

Then sign in, and either invite people from the People page or let them use **Request access** on the sign-in page and approve them under **Admin** → **Approvals**.

7 · Start on boot

```
sudo cp /opt/cis/deploy/ec2/cis.service /etc/systemd/system/  
sudo systemctl daemon-reload  
sudo systemctl enable --now cis  
sudo systemctl status cis
```

Without this, a reboot leaves the site down until you SSH in. Reboot once now to prove it works — better to find out today than during an incident.

8 · Backups

On a single instance the database lives on one EBS volume. Backups are entirely your responsibility, and an untested backup is a hope, not a backup.

```
# optional off-box copies
export S3_BUCKET=s3://your-bucket/cis      # needs an instance role with s3:PutObject

sudo crontab -e
15 2 * * * S3_BUCKET=s3://your-bucket/cis /opt/cis/deploy/ec2/backup.sh >> /var/log/cis-backup.log
2>&1
```

Also enable **EBS snapshots** via Data Lifecycle Manager — a daily snapshot with 7-day retention protects you against losing the whole volume, which a SQL dump does not.

Restore drill — do this once, before you need it

```
gunzip -c /var/backups/cis/cis-YYYYMMDD-HHMMSS.sql.gz \
| docker compose -f docker-compose.prod.yml exec -T postgres \
psql -U cis_app -d cis
```

Time it. However long that takes is your real recovery time, whatever your plan says.

9 · Updating

```
cd /opt/cis
git pull
docker compose -f deploy/ec2/docker-compose.prod.yml up -d --build
docker image prune -f
```

Migrations run automatically before the API starts, because the API depends on the `migrate` service completing successfully.

This causes roughly 30–90 seconds of downtime

while images rebuild and containers restart. That is the honest trade-off of a single box. Take a backup first if the release includes a migration:

```
/opt/cis/deploy/ec2/backup.sh && git pull && docker compose ... up -d --build
```

Rolling back

```
git log --oneline -5
git checkout <previous-sha>
docker compose -f deploy/ec2/docker-compose.prod.yml up -d --build
```

Rolling back code does not reverse a migration. Keep migrations backwards compatible: add columns nullable, backfill, make them required in a later release.

10 · Monitoring

```
sudo apt install -y amazon-cloudwatch-agent # or use the SSM package
```

The three that actually matter on one box:

Alarm	Threshold	Why
Disk used	> 80%	A full disk stops Postgres writing and is the most common single-box outage
StatusCheckFailed	≥ 1	The instance itself is unhealthy
Memory used	> 90%	Precedes the OOM killer taking a container

Add an external uptime check against `https://YOUR_DOMAIN/api/v1/health/ready`. A monitor running on the same box tells you nothing when the box is the problem.

```
docker compose -f docker-compose.prod.yml logs -f --tail=100 api
docker stats --no-stream
df -h && free -h
```

11 · Troubleshooting

Symptom	Cause and fix
No certificate; Caddy logs an ACME failure	DNS does not resolve to this instance yet, or port 80 is closed. Check <code>dig +short YOUR_DOMAIN</code> and the security group.
Site loads, every API call 404s	The domain in <code>Caddyfile</code> was never changed, so the site block does not match your host.
API container restarts in a loop	Env validation failed at boot. <code>logs api</code> names the missing variable on the first line.
Login works, all lists are empty	Migrations never ran — the <code>migrations/</code> folder is missing from the repo. §2.
<code>migrate</code> exits non-zero	Wrong <code>DATABASE_URL</code> , or the password differs between <code>POSTGRES_PASSWORD</code> and the URL.
Build killed around 80%	Out of memory. Add swap (§4) or use a larger instance.
Invitations never arrive	<code>EMAIL_DRIVER</code> still <code>console</code> , or SES is still in sandbox mode.
Everything died after a reboot	The systemd unit was never enabled. §7.
Disk full	<code>docker system prune -a</code> , then check <code>/var/lib/docker</code> and old backups.

12 · Cost

Item	Monthly
------	---------

t3.small on-demand	~\$15
30 GB gp3	~\$2.50
Elastic IP (while attached)	free
Snapshots and S3 backups	~\$2
Data transfer out	~\$1–5
Total, all-in-one	~\$21–25
Add RDS db.t4g.micro (shape B)	+~\$15

ap-south-1 on-demand, mid-2026 list prices. A one-year Savings Plan takes roughly 30% off the instance. Verify with the [AWS pricing calculator](#) before committing.

13 · When to move off a single instance

- Downtime during deploys stops being acceptable to your users.
- You need more than one instance for availability, not capacity.
- Losing the box for an hour would cost more than the difference in AWS bill.
- The database has grown past what you are willing to back up by hand.

The migration path is gentle: move the database to RDS first (shape B — one line in the compose file and one URL), then move the containers to ECS using the [AWS guide](#). The images and environment variables do not change.